Taller 02 — Afianzando los seis criterios técnicos de la Semana 3 (versión del profesor)

Versión PROFESOR (soluciones y guía de facilitación)

Notas generales para el profesor:

- Tiempo sugerido: 75-90 min en total.

Parte	Tema (criterio)	Tiempo
0	Calentamiento: ¿qué framework es?	5 min
1	Preprocesamiento (errores de formato y escalado)	20 min
2	Interpretabilidad (`coef_` , árbol, conteo de parámetros)	15 min
3	Rendimiento y facilidad de entrenamiento (cronometrar, GPU)	15 min
4	Despliegue (guardar y recargar)	10 min
5	Integrador y pregunta de cierre	10-15 min

- El código de la versión del profesor se ejecutó completo, sin errores, con `scikit-learn 1.9.1` , `tensorflow 2.21.0` (Keras 3) y `torch 2.15.0+cpu` en Windows 11 (CPU). Si el curso usa otras versiones, las cifras pueden variar levemente.
- Idea pedagógica central: cada parte trabaja **un criterio** de los seis de la Semana 3 con evidencia que el estudiante mide por sí mismo, para que la comparación entre *frameworks* no quede solo en teoría.
- Las tres celdas del Ejercicio 1.1 **fallan a propósito** en la versión del estudiante: el objetivo es que lean el mensaje de error y corrijan los datos.
- Los tiempos del Ejercicio 3.1 dependen del equipo; pide comparar el **orden de magnitud**, no los segundos exactos. Las semillas fijas dan resultados parecidos, no idénticos entre equipos.
- Es normal que `tf.config.list_physical_devices("GPU")` devuelva `[]` y `torch.cuda.is_available()` devuelva `False` ; no es un error de instalación.
- El Ejercicio 4.1 crea los archivos `modelo_sk.joblib` y `modelo_pt.pt` en la carpeta de trabajo; puedes pedir a los estudiantes que los borren al terminar.
- Continúa el Taller 01 y la pregunta orientadora del foro de la Semana 3.

Instrucciones generales:

- Trabaja en un notebook de Jupyter (Colab o local) con `scikit-learn` , `tensorflow` y `torch` instalados.

- Donde veas `# TU CÓDIGO AQUÍ`, reemplázalo por tu solución.
- Responde las preguntas en una celda de texto (Markdown), no como comentario de código.
- Duración estimada: 60-90 minutos.
- Este taller refuerza lo visto en el Taller 01 y en las diapositivas: **preprocesamiento, soporte GPU/rendimiento, facilidad de entrenamiento, interpretabilidad, comunidad/documentación e integración para despliegue**. Cada parte trabaja un criterio con evidencia medida por ti.

Parte 0 — Calentamiento: ¿qué framework es?

Escribe el nombre del *framework* (scikit-learn, TensorFlow/Keras o PyTorch) que corresponde a cada descripción:

#	Descripción	Framework
a	Todo el entrenamiento ocurre en una llamada a `fit`; no hay que declarar capas.	
b	Hay que escribir `zero_grad()`, `backward()` y `step()` en un bucle.	
c	Se define el modelo con `Sequential`, se llama a `compile` y luego a `fit`.	
d	Los datos deben llegar como tensores `float32`.	
e	El modelo se guarda con `joblib.dump`.	
f	El modelo se guarda con `model.save("x.keras")`.	

Respuestas: a = scikit-learn · b = PyTorch · c = TensorFlow/Keras · d = PyTorch · e = scikit-learn · f = TensorFlow/Keras.

Parte 1 — Preprocesamiento: cada framework pide los datos a su manera

Ejercicio 1.1 — Solución — Tres errores, tres arreglos

Cada celda siguiente **falla** porque los datos no tienen el formato que el *framework* espera. Lee el mensaje de error y corrige solo los datos (no el modelo).

a) scikit-learn espera una matriz 2D (filas = muestras, columnas = variables).

In [1]:
```
from sklearn.linear_model import LinearRegression

X = [1, 2, 3, 4]          # <- 1D: provoca el error
```

```
y = [2, 4, 6, 8]

X = [[1], [2], [3], [4]]  # solución: una columna por variable
modelo = LinearRegression().fit(X, y)
print(modelo.predict([[5]]))
```

```
[10.]
```

b) Keras (Keras 3) no acepta listas de Python: exige arreglos de NumPy.

In [2]:
```
import numpy as np
import tensorflow as tf

X = [1.0, 2.0, 3.0, 4.0]  # <- lista: provoca el error
y = [2.0, 4.0, 6.0, 8.0]

X = np.array(X)           # solución
y = np.array(y)

m = tf.keras.Sequential([tf.keras.layers.Input(shape=(1,)), tf.keras.layers.Dense(1
m.compile(optimizer="sgd", loss="mse")
m.fit(X, y, epochs=5, verbose=0)
print("Keras entrenó sin error")
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
Keras entrenó sin error
```

c) PyTorch exige tensores y el mismo tipo de dato que los pesos (`float32`). Un arreglo de NumPy trae `float64` por defecto.

In [3]:
```
import numpy as np
import torch
import torch.nn as nn

X = np.array([[1.0], [2.0], [3.0], [4.0]])   # <- float64 y ndarray: provoca el err
modelo = nn.Linear(1, 1)

X = torch.tensor(X, dtype=torch.float32)  # solución

print(modelo(X).shape)
```

```
torch.Size([4, 1])
```

Ejercicio 1.2 — Solución — Escalar sin hacer trampa

Con `load_digits` (sin dividir entre 16 esta vez), estandariza los píxeles con `StandardScaler` . **Regla de oro:** el escalador se ajusta (`fit`) solo con los datos de entrenamiento y luego se aplica (`transform`) a los de prueba.

In [4]:
```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression

X, y = load_digits(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

esc = StandardScaler().fit(X_train)          # fit SOLO con entrenamiento
X_train_s = esc.transform(X_train)
X_test_s = esc.transform(X_test)

clf = LogisticRegression(max_iter=1000).fit(X_train_s, y_train)
print("media train (≈0):", X_train_s.mean().round(3))
print("accuracy test:", round(clf.score(X_test_s, y_test), 4))
```

```
media train (≈0): -0.0
accuracy test: 0.9722
```

Salida esperada: media de entrenamiento ≈ 0 y `accuracy` alrededor de 0.97 (varía levemente según la versión).

Pregunta de reflexión 1: ¿Por qué es un error ajustar el `StandardScaler` con *todos* los datos antes de dividir en entrenamiento y prueba? ¿Qué parte de la prueba "se filtra" al modelo?

Respuesta esperada: si el escalador ve los datos de prueba, la media y la desviación calculadas incluyen información de ellos (*fuga de datos*, *data leakage*). El modelo termina evaluado con datos que ya influyeron en su preparación y el accuracy sale más optimista que el real. Se simula el caso real: al desplegar, los datos nuevos no existen todavía.

Parte 2 — Interpretabilidad: ¿puedo explicar la decisión?

Ejercicio 2.1 — Solución — Leer un modelo lineal

Entrena `LinearRegression` con `y = 2x` y lee sus parámetros con `coef_` e `intercept_`.

In [5]:
```
from sklearn.linear_model import LinearRegression

X = [[1], [2], [3], [4]]
y = [2, 4, 6, 8]

m = LinearRegression().fit(X, y)
print("w =", m.coef_[0], " b =", round(m.intercept_, 6))
```

```
w = 2.0  b = 0.0
```

Salida esperada: `w = 2.0  b = 0.0`. El modelo **se lee directamente**: la fórmula es `y = 2.0·x + 0.0`.

Ejercicio 2.2 — Solución — Un árbol de decisión sobre `iris`

Entrena un `DecisionTreeClassifier(max_depth=3, random_state=42)` con `iris`, imprime sus reglas con `export_text` y las importancias de cada variable.

In [6]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)

arbol = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_train, y_train)
print("accuracy:", arbol.score(X_test, y_test))
print(export_text(arbol, feature_names=iris.feature_names))
for nombre, imp in zip(iris.feature_names, arbol.feature_importances_):
    print(f"{nombre}: {imp:.3f}")
```

```
accuracy: 1.0
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.65
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.65
|   |   |   |--- class: 2
|   |--- petal length (cm) >  4.75
|   |   |--- petal width (cm) <= 1.75
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.75
|   |   |   |--- class: 2

sepal length (cm): 0.000
sepal width (cm): 0.000
petal length (cm): 0.935
petal width (cm): 0.065
```

Salida esperada: accuracy 1.0 en esta partición; las reglas usan casi solo las variables de **pétalo** (la primera es `petal length (cm) <= 2.45` → setosa) y las importancias de los sépalos son ≈ 0.

Ejercicio 2.3 — Solución — Contar parámetros de una red

Cuenta los parámetros de una red pequeña de PyTorch (`64 → 32 → 10`) y compáralos con los de la regresión logística de scikit-learn sobre `digits` (que tiene `coef_` e `intercept_`).

In [7]:
```
import torch.nn as nn
from sklearn.datasets import load_digits
```

```
from sklearn.linear_model import LogisticRegression

red = nn.Sequential(nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 10))

X, y = load_digits(return_X_y=True)
reg = LogisticRegression(max_iter=1000).fit(X / 16.0, y)

n_red = sum(p.numel() for p in red.parameters())
n_reg = reg.coef_.size + reg.intercept_.size
print("parámetros de la red:", n_red)
print("parámetros de la regresión logística:", n_reg)
```

```
parámetros de la red: 2410
parámetros de la regresión logística: 650
```

Salida esperada: red = **2 410** parámetros (64·32 + 32 + 32·10 + 10); regresión logística = **650** (10·64 + 10).

Pregunta de reflexión 2: Con el árbol de `iris` puedes decir "es versicolor porque el largo del pétalo es mayor que 2.45 y …". ¿Podrías dar una explicación igual de directa con la red de 2 410 parámetros? ¿En qué tipo de problema aceptarías perder esa explicación y en cuál no?

Respuesta esperada: no; los pesos de la red están distribuidos en miles de números sin lectura individual. Se acepta perder interpretabilidad cuando lo que importa es el desempeño y el costo de un error es bajo (reconocer imágenes, recomendaciones); no cuando hay que justificar la decisión a una persona o a un regulador (crédito, salud). Debe conectar con el criterio *interpretabilidad*.

Parte 3 — Rendimiento y facilidad de entrenamiento: mide, no supongas

Ejercicio 3.1 — Solución — Cronometrar tres formas de entrenar `digits`

Mide con `time.perf_counter()` cuánto tarda cada entrenamiento sobre `digits` (80 % entrenamiento) y su accuracy: **KNN** (scikit-learn), **red de Keras** (64-32-10, Adam, 20 épocas, `verbose=0`) y **red de PyTorch** (misma arquitectura, bucle propio, 20 épocas, lotes de 32).

In [8]:
```
import time
import numpy as np
import torch
import torch.nn as nn
import tensorflow as tf
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

```
torch.manual_seed(0); tf.keras.utils.set_random_seed(0)

X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype("float32")
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

resultados = {}

# --- scikit-learn: KNN
t0 = time.perf_counter()
knn = KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)
resultados["KNN (sklearn)"] = (time.perf_counter() - t0, knn.score(X_test, y_test))

# --- Keras
t0 = time.perf_counter()
k = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(64,)),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax"),
])
k.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accur
k.fit(X_train, y_train, epochs=20, verbose=0)
resultados["Red (Keras)"] = (time.perf_counter() - t0, k.evaluate(X_test, y_test, v

# --- PyTorch
t0 = time.perf_counter()
red = nn.Sequential(nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 10))
opt = torch.optim.Adam(red.parameters())
perdida = nn.CrossEntropyLoss()
Xt, yt = torch.tensor(X_train), torch.tensor(y_train, dtype=torch.long)
for _ in range(20):
    perm = torch.randperm(len(Xt))
    for i in range(0, len(Xt), 32):
        idx = perm[i:i + 32]
        opt.zero_grad()
        perdida(red(Xt[idx]), yt[idx]).backward()
        opt.step()
with torch.no_grad():
    acc = (red(torch.tensor(X_test)).argmax(1).numpy() == y_test).mean()
resultados["Red (PyTorch)"] = (time.perf_counter() - t0, acc)

for nombre, (seg, acc) in resultados.items():
    print(f"{nombre:15s} tiempo = {seg:7.3f} s   accuracy = {acc:.3f}")
```

```
KNN (sklearn)   tiempo =   0.001 s   accuracy = 0.983
Red (Keras)     tiempo =   1.889 s   accuracy = 0.964
Red (PyTorch)   tiempo =   1.988 s   accuracy = 0.953
```

Salida esperada (los tiempos dependen del equipo): KNN en milisegundos y accuracy ≈ 0.98; las dos redes en el orden de 1-3 s y accuracy ≈ 0.95-0.97. Lo importante es el **orden de magnitud**: el modelo clásico entrena mucho más rápido y no pierde en este problema pequeño.

Ejercicio 3.2 — Solución — ¿Hay GPU disponible?

Consulta los dispositivos que ve cada *framework*: `tf.config.list_physical_devices("GPU")` y `torch.cuda.is_available()`.

In [9]:
```
import tensorflow as tf
import torch

print("TensorFlow GPUs:", tf.config.list_physical_devices("GPU"))
print("PyTorch CUDA:", torch.cuda.is_available(), "| build CUDA:", torch.version.cu
```

```
TensorFlow GPUs: []
PyTorch CUDA: False | build CUDA: None
```

Salida esperada en un portátil sin GPU o en Windows nativo: `[]` y `False` (con `build CUDA: None` si la instalación es CPU). **No es un error.** TensorFlow ≥ 2.11 no usa GPU en Windows nativo (se usa WSL2 o Colab); en PyTorch depende de la *build* instalada.

Pregunta de reflexión 3: Con tus tiempos del Ejercicio 3.1, ¿justifica la red neuronal su costo para `digits`? ¿Qué tendría que cambiar (datos, tamaño, hardware) para que sí valiera la pena?

Respuesta esperada: en `digits` (1 797 imágenes de 8×8) el KNN o la regresión logística igualan o superan a la red y entrenan decenas o miles de veces más rápido, por lo que la red no se justifica. Valdría la pena con muchos más datos, imágenes más grandes o a color, audio/texto, o cuando se dispone de GPU. Conecta con *GPU/rendimiento* y con la regla práctica: **primero una línea base clásica**.

Parte 4 — Despliegue: guardar, recargar y predecir igual

Ejercicio 4.1 — Solución — Un modelo que sobrevive a reiniciar el kernel

Entrena el modelo `y = 2x` en scikit-learn y en PyTorch, guárdalos en disco, recárgalos y comprueba que la predicción para `X = 5` es **idéntica** antes y después.

In [10]:
```
import os
import joblib
import torch
import torch.nn as nn
from sklearn.linear_model import LinearRegression

# --- scikit-learn
sk = LinearRegression().fit([[1], [2], [3], [4]], [2, 4, 6, 8])

# --- PyTorch
torch.manual_seed(0)
X = torch.tensor([[1.0], [2.0], [3.0], [4.0]])
```

```
y = torch.tensor([[2.0], [4.0], [6.0], [8.0]])
pt = nn.Linear(1, 1)
opt = torch.optim.SGD(pt.parameters(), lr=0.01)
for _ in range(200):
    opt.zero_grad()
    nn.MSELoss()(pt(X), y).backward()
    opt.step()

# Guardar
joblib.dump(sk, "modelo_sk.joblib")
torch.save(pt.state_dict(), "modelo_pt.pt")

# Recargar
sk2 = joblib.load("modelo_sk.joblib")
pt2 = nn.Linear(1, 1)                        # hay que reconstruir la arquitectura
pt2.load_state_dict(torch.load("modelo_pt.pt", weights_only=True))

print("sklearn  antes/después:", sk.predict([[5]]), sk2.predict([[5]]))
print("pytorch  antes/después:", pt(torch.tensor([[5.0]])).item(), pt2(torch.tensor
print("tamaños (bytes):", os.path.getsize("modelo_sk.joblib"), os.path.getsize("mod
```

```
sklearn  antes/después: [10.] [10.]
pytorch  antes/después: 9.579675674438477 9.579675674438477
tamaños (bytes): 639 1909
```

Salida esperada: predicciones iguales antes/después en ambos (≈ 10 en scikit-learn; ≈ 9.6-10 en PyTorch según la semilla) y archivos de unos cientos de bytes a ~2 KB. Observación clave: PyTorch guarda solo los **pesos**, por eso hay que reconstruir la arquitectura al recargar.

Pregunta de reflexión 4: ¿Qué diferencia práctica hay entre `joblib.dump(modelo, ...)` y `torch.save(modelo.state_dict(), ...)` al momento de recargar el modelo en otro programa?

Respuesta esperada: `joblib` guarda el objeto completo (algoritmo y parámetros) y se recarga con una línea. `state_dict` guarda solo los pesos: el otro programa necesita el código que define la arquitectura para reconstruirla antes de cargar los pesos. Conecta con *integración para despliegue*.

Parte 5 — Integrador: elegir con argumentos

Para cada escenario, elige un *framework* de primera opción y justifica con **al menos un criterio** de los seis (preprocesamiento, GPU/rendimiento, facilidad de entrenamiento, interpretabilidad, comunidad/documentación, despliegue).

#	Escenario	Primera opción	Criterio y justificación
1	Un banco debe explicar por qué niega un crédito, con una tabla de 5 000 clientes.		

#	Escenario	Primera opción	Criterio y justificación
2	Clasificar millones de fotos de productos y entrenar con GPU.		
3	Investigar una arquitectura nueva y depurar cada paso del entrenamiento.		
4	Publicar en una API pequeña un modelo de predicción de precios con datos tabulares.		
5	Llevar un modelo a una aplicación móvil o al navegador.		

Respuestas de referencia (admiten variantes bien argumentadas):

#	Primera opción	Criterio
1	scikit-learn (regresión logística o árbol)	Interpretabilidad: `coef_` / reglas del árbol permiten justificar cada decisión.
2	TensorFlow/Keras o PyTorch	GPU/rendimiento: las redes con GPU son su terreno natural.
3	PyTorch	Facilidad de entrenamiento (control): cada paso del bucle es visible y modificable.
4	scikit-learn + `joblib`	Despliegue: archivo pequeño, dependencias livianas, integra fácil con Flask/FastAPI.
5	TensorFlow (LiteRT, TF.js)	Despliegue: ecosistema para móvil y navegador.

Pregunta de cierre: Con lo que mediste en este taller, completa la frase y justifícala en 3-4 líneas: "*Antes de usar una red neuronal, yo primero…*"

Respuesta esperada: entrenar un modelo clásico como línea base (por ejemplo regresión logística, KNN o un árbol) y comparar exactitud, tiempo e interpretabilidad. Solo si el modelo clásico no alcanza el desempeño necesario, o los datos son imágenes/audio/texto a gran escala, se justifica la complejidad de una red.

Cierre sugerido: no existe un *framework* "mejor" en abstracto; la elección depende del problema, los datos y el contexto de aplicación.